Assignment 3.A (3%) - RESTful API - Read Functionality

T-213-VEFF, Web programming I, 2026-1

Reykjavik University - Department of Computer Science, Menntavegi 1, IS-101 Reykjavík, Iceland

Deadline: Friday, March 6th 2026, 23:59

This is Part A of the third assignment in Web Programming I. In this part, you will implement the read functionality for a REST API using Express. You will work with the provided starter file and complete a set of event and attendee endpoints according to the specifications. **This assignment should be completed in group of 2-3 students.**

1 Read API Functionality

In Part A, you will implement the read functionality of the Events API. The focus of this part is correct route definition, parameter handling, filtering, and returning appropriate HTTP status codes and JSON responses. You are also expected to compute derived data where required. You may need to review array methods, Express routing and HTTP status codes beyond what has been covered in lectures. Helpful resources include https://expressjs.com/en/guide/routing.html, https://expressjs.com/en/5x/api.html and https://www.w3schools.com/js/js_array_methods.asp.

2 Starting the project

2.1 Starter Pack

The supplementary material contains a sample Node.js project with a `package.json`, an `index.js`, and a `data/initialData.js` file. The `index.js` file already includes the Express setup, middleware configuration, base path variables, and server initialization. The `data/initialData.js` file contains the initial data for events and attendees, as well as id counters. You must implement all required functionality in `index.js`. No other provided files may be modified or added, and the server initialization code at the end of the file must not be removed or changed. The

required dependencies (including Express and CORS) are already defined in the `package.json` file. You can install them by running: `npm install` in your project directory.

2.2 Starting the backend project in "development mode"

1. Navigate to the **assignment3_starter_pack** folder in a terminal window.

2. Install the NPM packages listed in ***package.json*** by running ***npm install*** command. This will fetch all the required packages needed to run the project.

3. To start up the process, you can run the ***npm run dev*** command to start the process in developer mode. Which in our case, means that the process will be watching ***index.js*** for changes. When the code is updated, the process will automatically restart to reflect the changes made to the code. The process will print out a message telling your it has started:

```
> assignment_3@0.0.1 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter ‘rs‘
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting ‘node index.js‘
Server running on http://localhost:3000
```

4. Your backend is now running http://localhost:3000 and can be accessed via Insomnia, Postman and cURL.

3 Resources

This project defines two resource types: **events** and **attendees**. An attendee may be registered for multiple events.
An **event** has the following attributes:

- ***id*** - A unique integer identifying the event.
- ***name*** - The name of the event (string).
- ***location*** - The location of the event (string).
- ***date*** - The date of the event (string, format YYYY-MM-DD).

An **attendee** has the following attributes:

- ***id*** - A unique integer identifying the attendee.
- ***name*** - The name of the attendee (string).
- ***email*** - The email of the attendee (string).
- ***eventIds*** - An array of event ids the attendee is registered for.

You are encouraged to implement these resources in your backend as you see fit. However, it's essential that the backend can return and manage these attributes in the specified format.

4 Endpoints

The following endpoints shall be implemented for the **events**.

1. **Read** all events

   Returns an array of all events. For each event, the id, name, location, and date are included in the response. The endpoint supports optional query parameters `name` and `location`. Filtering is case-insensitive and matches events where the given value is contained in the corresponding field. If both query parameters are provided, both filters are applied. Only these query parameters are allowed; any other query parameter results in a 400 response. If no events match, an empty array is returned.

2. **Read** a specific event

   Returns a specific event with its attributes (id, name, location, and date), as well as an additional attribute `attendeeCount`, indicating how many attendees are registered for the event. The endpoint accepts an `eventId` as part of the path, which must be an integer. If the id is invalid, the request returns 400. If the event is not found, the request returns 404. If successful, the event is returned with status 200.

The following endpoint shall be implemented for the **attendees**.

1. **Read** all attendees

   Returns an array of all attendees (including all attributes). If no attendees exist, an empty array is returned. The request returns status 200.

5 Requirements

The following requirements shall be followed:

1. The application shall adhere to REST principles and best practices discussed in the lectures.
2. Resource collections shall use plural nouns (e.g., `/events`, `/attendees`). Specific resources shall be addressed using their id as part of the URL.
3. JSON shall be used as the request and response format.
4. Appropriate HTTP status codes shall be used. In particular:
   - 200 for successful GET requests.
   - 400 if the request is invalid (e.g., invalid id format or unsupported query parameters).
   - 404 if a requested resource does not exist or a non-existing endpoint is called.
   - In all error cases (e.g., 400, 404), the response body shall contain a JSON object with a descriptive `message` field explaining the error.
5. The backend shall be served at `http://localhost:3000/api/v1/`. Do not change the port.
6. The application shall be implemented as a Node.js application using the provided starter project. You are not allowed to modify the `package.json`.

7. The application shall be started using the command `npm start`.
8. Only built-in Node.js modules, Express, and CORS may be used. No additional libraries are permitted.
9. No persistence (database, file storage, etc.) may be added.
10. The code shall follow basic JavaScript and Express best practices:
    - Use `const` and `let` (do not use `var`).
    - Use arrow functions.
    - Keep route handlers readable and well-structured.
    - Avoid unnecessary code duplication.
    - Use built-in array methods (e.g., `map`, `filter`, `find`) instead of manual loops where appropriate.

6 Submission

The project is submitted via Gradescope. Submission should **only** contain one file:

- index.js

Submissions will not be accepted after the deadline.

Grading and point deduction

Assignment 3.A is graded fully automatically using Gradescope. The total score for Assignment 3.A is **30 points**. Submissions and the autograder will open on Wednesday, March 4th 2026. After each submission, you will receive a grade and detailed feedback from Gradescope. You are encouraged to use this feedback to improve your solution and may submit as many times as needed before the deadline.

Certain violations result in a score of **0 points** for this part, in which case grading stops immediately.

Criteria	Point deduction
Required file: The submission contains `index.js`.	Missing `index.js`: **-30 points (score reduced to 0; grading stops)**.
No extra files: Only `index.js` is submitted.	Any extra/unexpected file(s) included: **-30 points (score reduced to 0; grading stops)**.
Code must be testable automatically: The application must export the Express app correctly and run without modification in the provided test environment. The base path and server initialization code must remain unchanged.	Code cannot be executed or tested automatically (e.g., missing export, modified server initialization, changed base path): **-30 points (score reduced to 0; grading stops)**.
Read all events (10 points): Correct implementation of the endpoint that returns all events, including filtering functionality and appropriate validation and error handling.	Missing or incorrect implementation: up to **-10 points**, depending on severity.
Read a specific event (10 points): Correct implementation of the endpoint that returns a single event by id, including the computed attendeeCount, validation, and appropriate error handling.	Missing or incorrect implementation: up to **-10 points**, depending on severity.
Read all attendees (5 points): Correct implementation of the endpoint that returns all attendees, including appropriate response structure and status codes.	Missing or incorrect implementation: up to **-5 points**, depending on severity.
JavaScript best practices (5 points): Clean, consistent code style (e.g., `const`/`let`, no `var`, arrow functions, and readable structure).	Significant deviation from required practices: up to **-5 points**.